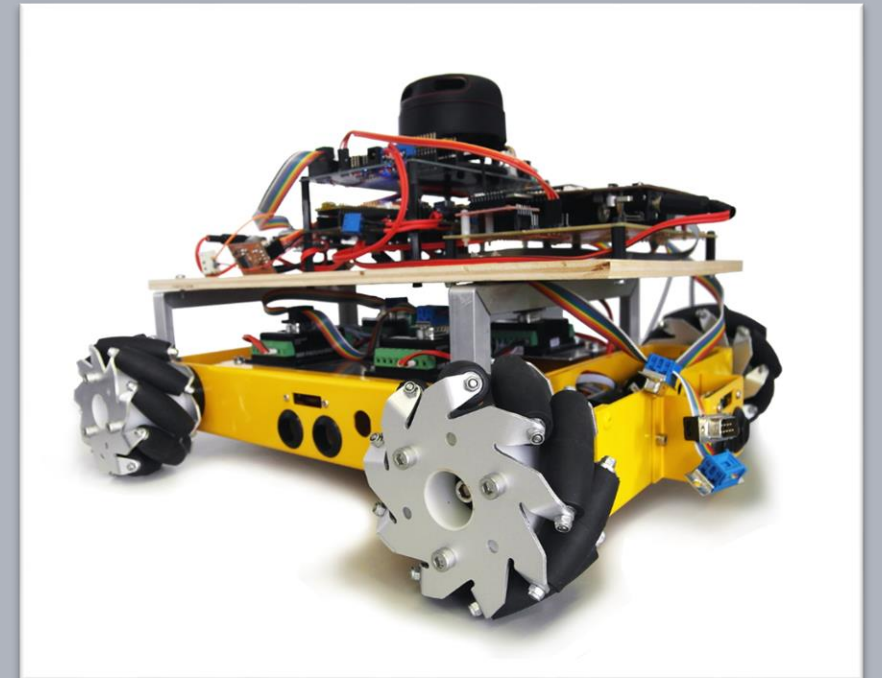


Advanced Embedded Architectures and Applications

Introduction

Prof. Dr.-Ing. Peter Fromm



Content

- A brief history of embedded systems
- The showcase / requirements
- Functional Architecture
- A first hardware architecture
- Open Issues and Risks
- A first software architecture

Definition Embedded System

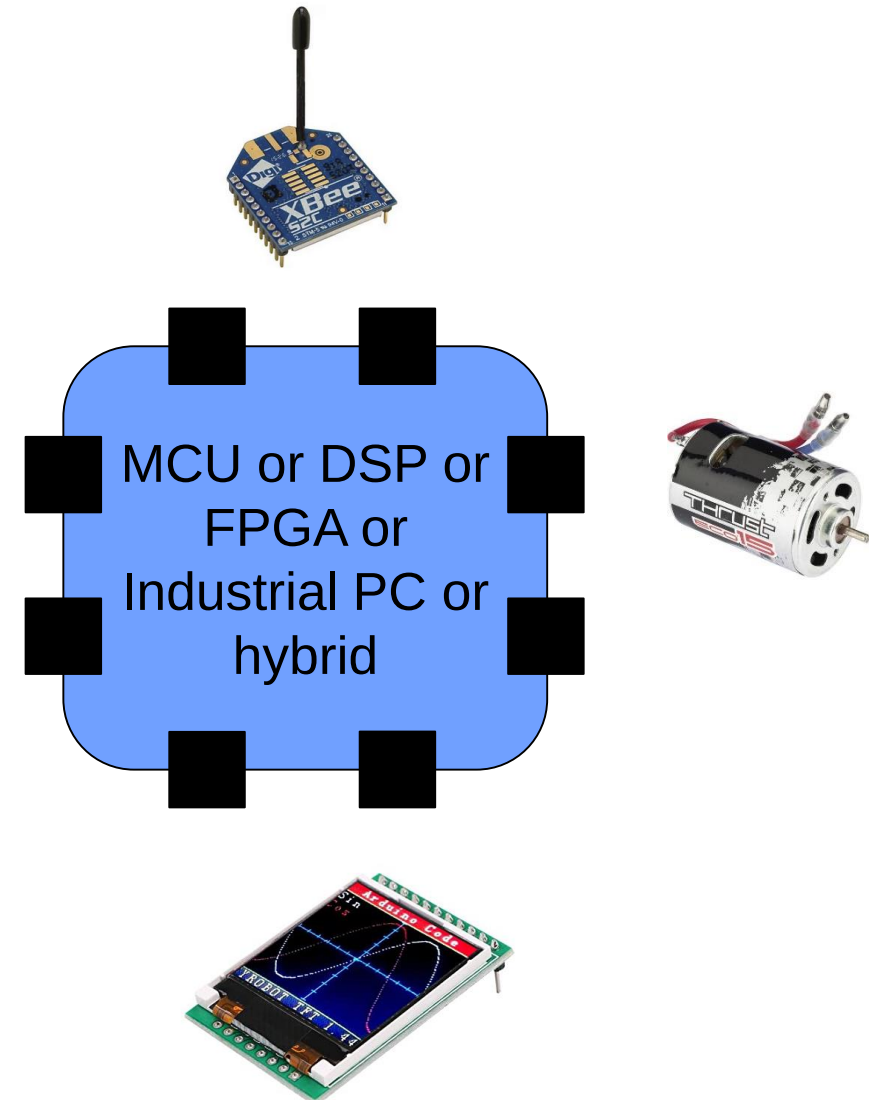
An embedded system is a computer system—a combination of a **computer processor**, computer memory, and input/output peripheral devices—that has a dedicated function within a **larger mechanical or electrical system**. It is embedded as part of a complete device often including electrical or electronic hardware and mechanical parts. Because an embedded system typically controls physical operations of the machine that it is embedded within, it often has real-time computing constraints. Embedded systems control many devices in common use today. **Ninety-eight percent of all microprocessors manufactured are used in embedded systems.**

Since the embedded system is dedicated to specific tasks, design engineers can optimize it to **reduce the size and cost of the product** and increase the **reliability and performance**.

Embedded systems range from portable devices such as digital watches and MP3 players, to large stationary installations like traffic light controllers, programmable logic controllers, and large complex systems like hybrid vehicles, medical imaging systems, and avionics. Complexity varies from low, with a single microcontroller chip, to very high with multiple units, peripherals and networks mounted inside a large equipment rack. (Wikipedia)

Structure of Embedded Systems

- Embedded Controller
- Peripheral Ports
- Sensors
- Actors
- Communication Channels
- HMI Elements
- Safety and Security playing an increasing role!

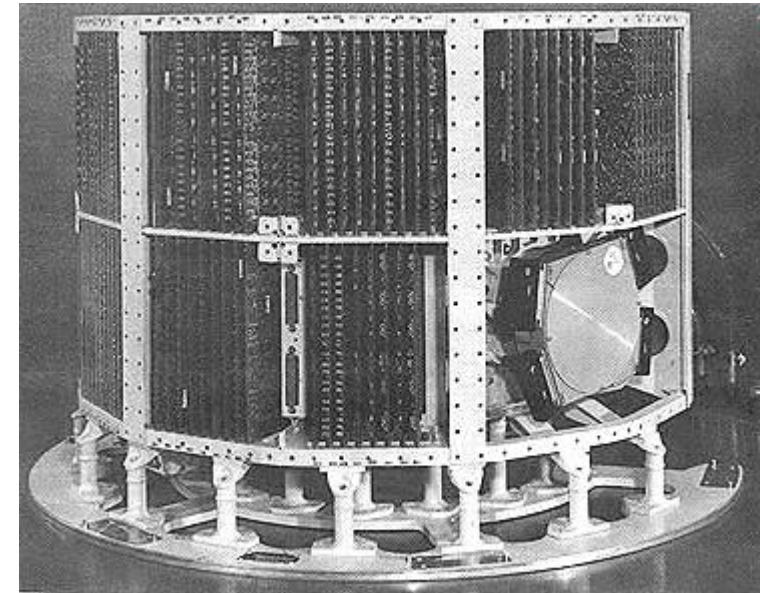


A little bit of history – the 60ies Apollo Guidance Computer

At the project's inception, the Apollo guidance computer was considered the riskiest item in the Apollo project as it employed the then newly developed monolithic integrated circuits to reduce the size and weight.



Minuteman Guidance Computer



History of Embedded Systems – 70ies

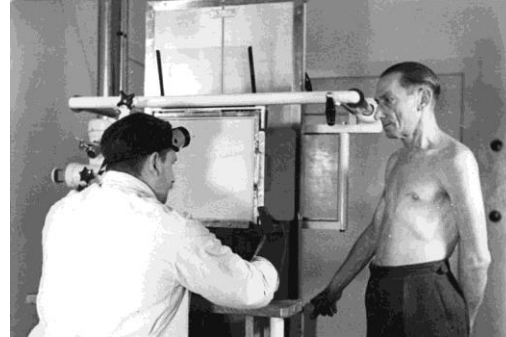
- One of the first “real” microcontroller was the Intel 8048, released in 1976.
- The 8048 has a Modified Harvard architecture, with internal or external program ROM and 64–256 bytes of internal (on-chip) RAM. The I/O is mapped into its own address space, separate from programs and data.
- It was used in the Magnavox Odyssey video game console, the Roland Jupiter-4 and Roland ProMars analog synthesizers, and (in its 8042 variant) in the original IBM PC keyboard.



History of Embedded Systems – 80ies

- In the 80ies and 90ies, embedded system were introduced in many areas of transportation (e.g. automotive, flight hardware) for improved control applications and networking. A major burst was achieved with the introduction of the CAN bus developed in 1983 at Robert Bosch GmbH.
- Today, consumer embedded systems include personal digital assistants (PDAs), MP3 players, mobile phones, videogame consoles, digital cameras, DVD players, GPS receivers, and printers. Many household appliances, such as microwave ovens, washing machines and dishwashers, are including embedded systems to provide flexibility, efficiency and features.

Examples of nowadays Embedded Systems

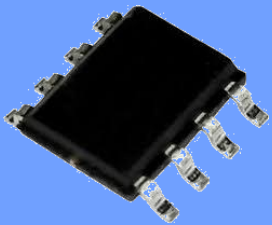


Various Embedded Controllers

IOT domain

- Small, cheap devices
- Low power consumption
- Limited memory (KB)
- Limited peripherals
- Bare Metal

PSOC4 M0



Realtime Control Applications

- Application Specific Hardware
- Softcore

XILINK Spartan FPGA

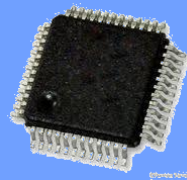


Realtime Control Applications

- Medium performance / price
- Low power consumption
- Medium memory (MB)
- Many peripherals
- RTOS (Autosar, FreeRTOS,...)

Infineon Aurix Multicore

STM32 M4



Mini / Single Board Computer

- High performance / price
- Higher power consumption
- Large memory (GB)
- Multicore
- Graphic capabilities
- Linux

NVIDIA Nano



Raspberry Pi



Which one has more computation power?



PSOC4 with ARM Cortex M0

Acorn Model B
My first home computer – 40
years ago



Where we are now...

Embedded Coding Training 2003

Why Safer Programming?

Evolution of the amount of software in cars:

- 1996: 20.000 lines of C code (80.000 lines of assembly language)
- 1999: 200.000 lines of C code
- 2002: estimated: >1.000.000 lines of C code

Some current cluster projects:

- PQ 35 86074* 308787 (B-Sample)
- Lancia LL/ML 15786 77921 (B-Sample)

GM 1999: Recall of 3.5 million cars because of software defect
Old SAT: 10-25 Mio Euro pa for recalls



*preprocessed/unprocessed LOC

C Programmings Guidelines - Slide 5

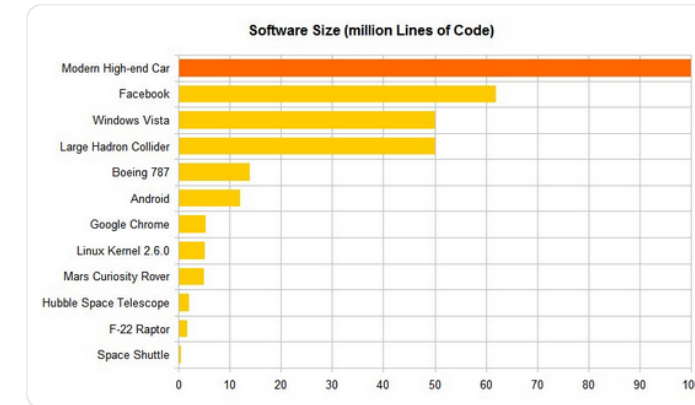
SV | IP | DT | DS | QE | Dr. P. Fromm



Rich Rogers
@RichRogersIoT

Folgen

"There are 100M lines of code in the average modern car. This should terrify any experienced programmer." - @JosephEames



15:00 - 29. Jan. 2018

1.556 Retweets 1.975 „Gefällt mir“-Angaben



92 1,6 Tsd. 2,0 Tsd.



Ryan Schmidt @rms80 · 30. Jan.

Antwort an @RichRogersIoT @ftrain @josepheames

Apparent source of this number... "probably" ... spectrum.ieee.org/transportation...

"it probably contains close to 100 million lines of software code," says Manfred Broy, a professor

Key Challenges

Nowadays embedded systems have the software size of desktop applications – strong need for architecture!

Many embedded systems are realized as open safety applications. Safety AND security becomes more focused.

Growing provision of libraries (of unknown quality)

The complexity of control algorithms is tremendously growing (AI,...). Classic hackaton approach will not work anymore.

Hardware becomes more complex (multicore, FPGA, hybrid)

Costs are still a major issue ;-)

Our showcase

First (rough) set of requirements

We want to develop a simplified version of the StudentCar.

- 4 DC Engines are driving 4 wheels
- Use a wireless communication for joystick control
- Provide elementary autonomous driving features
 - Obstacle detection
 - Dead reckoning positioning
- Consider a bit of safety
 - Detect broken communication link
- Support application reuse by separating application software (asw) from basic software / driver (bsw)



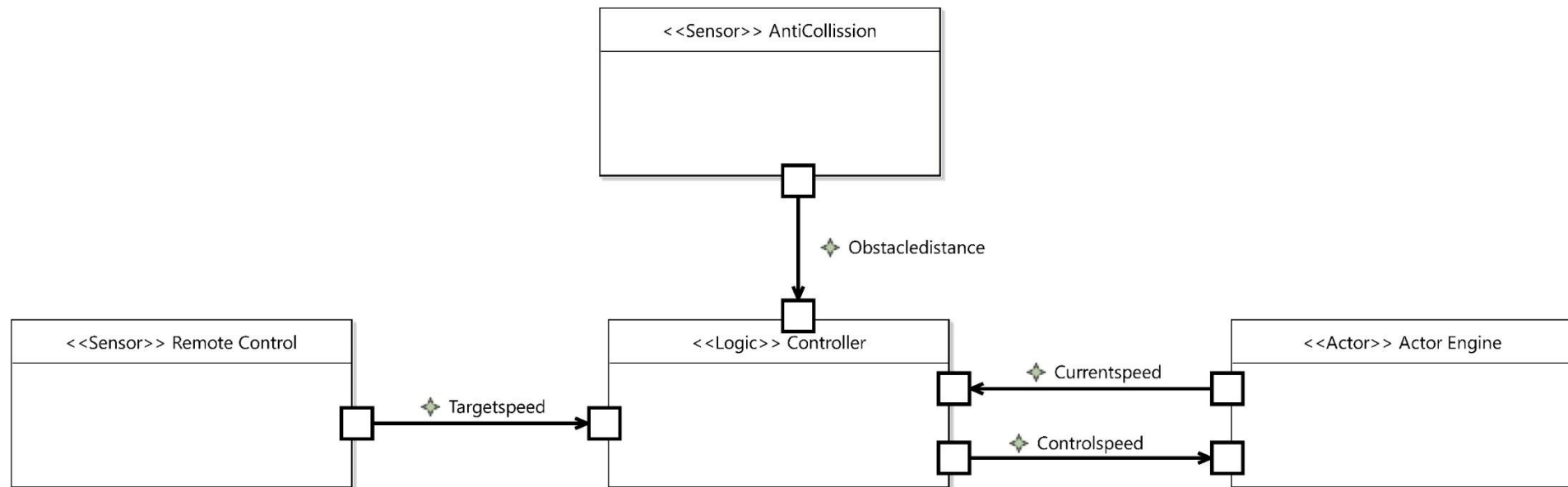
Let's design a first architecture

- What is the job of requirements, versus architecture versus hardware design versus code?
- What needs to be described in an architecture?
- How can we describe it?
- On which level of knowledge are we?
- Can we already create a class diagram?
- Or do we need something more abstract?

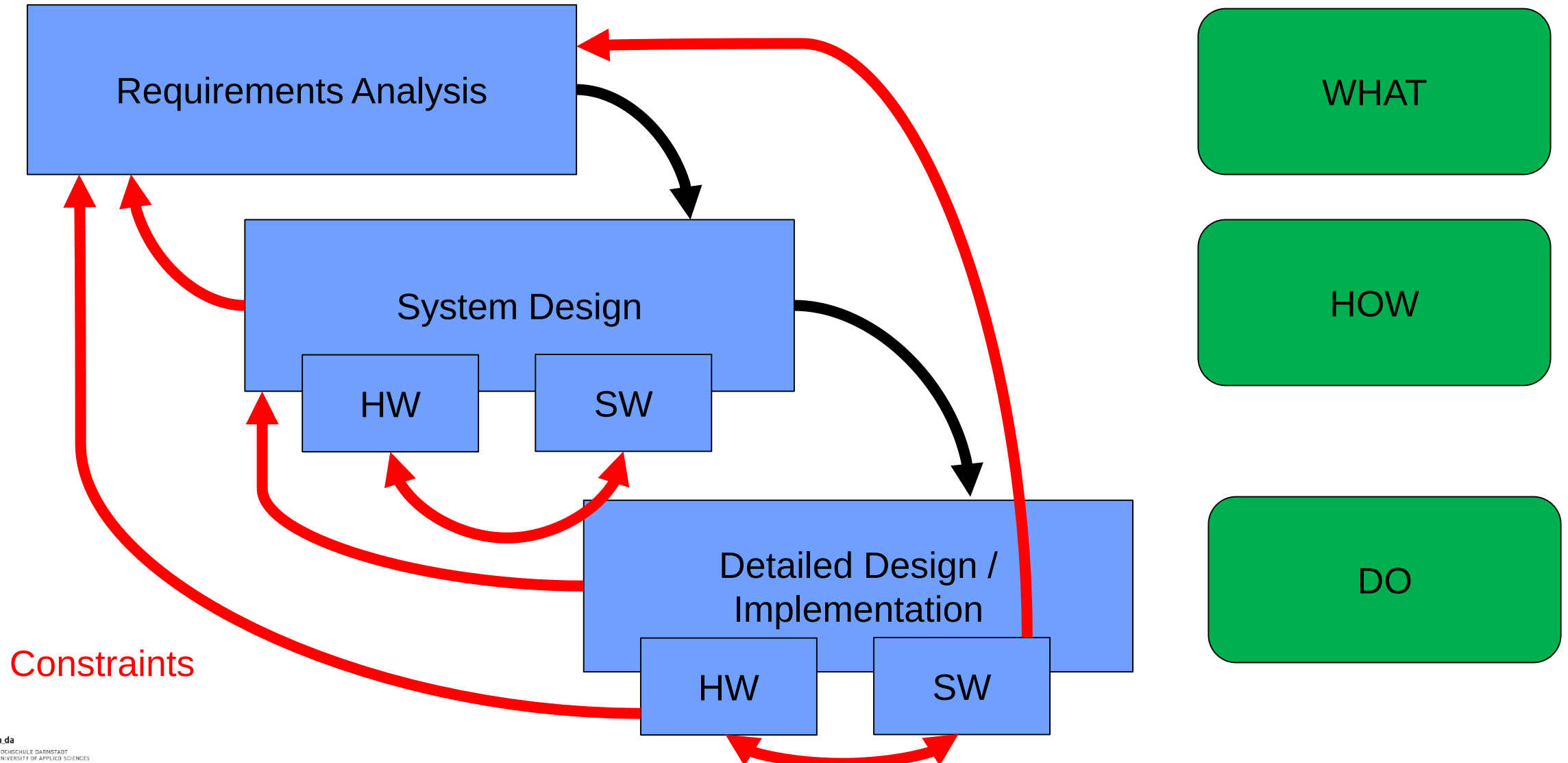


Functional Architecture

- A very simple form of a block diagram, focusing on the information flow.
- No decisions have been taken on hardware or software design yet.
- The goal is to get a first technical view on the system we want to develop.



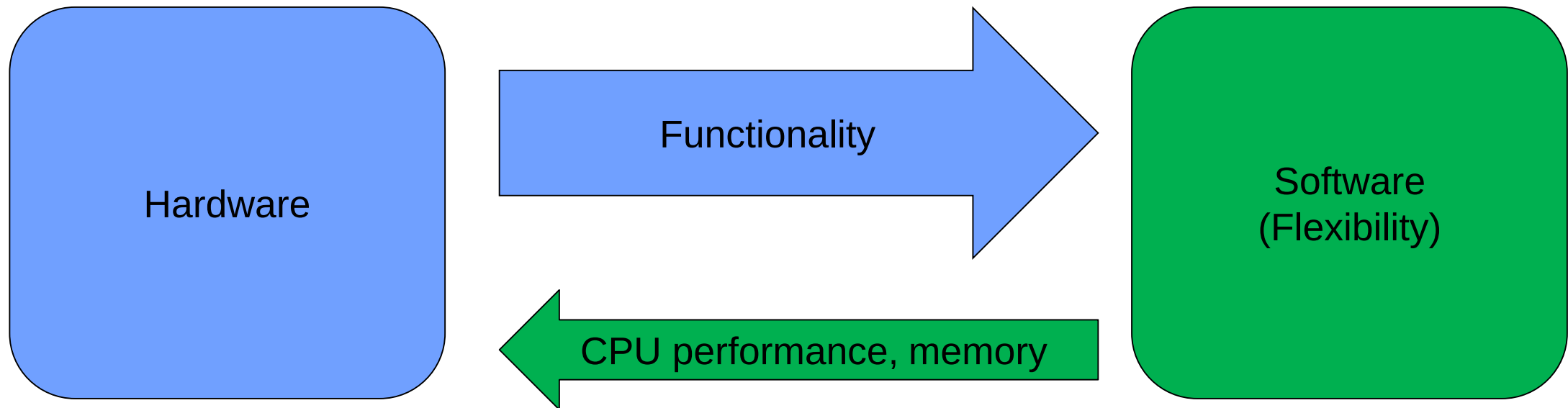
The myth of the V-Model (left side only)



System Engineering

We design a system, which means we are selecting / developing hardware AND software components.

- Hardware limitations may result in more complex software
- More complex software may require a stronger MCU
- Design freeze of hardware is usually way earlier than software – good idea?

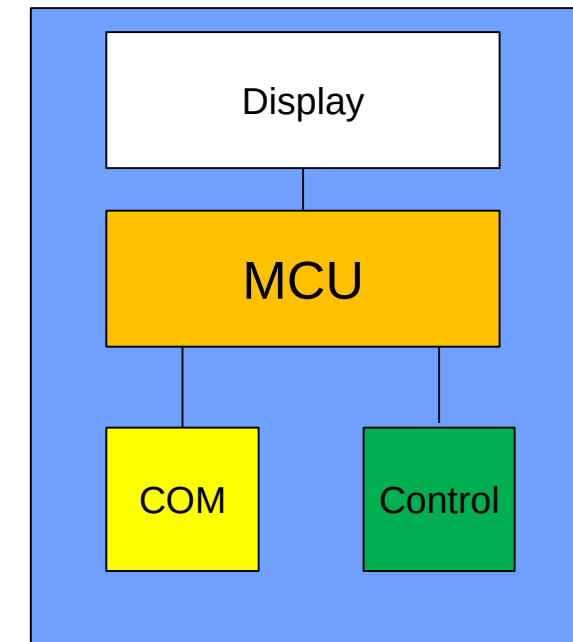
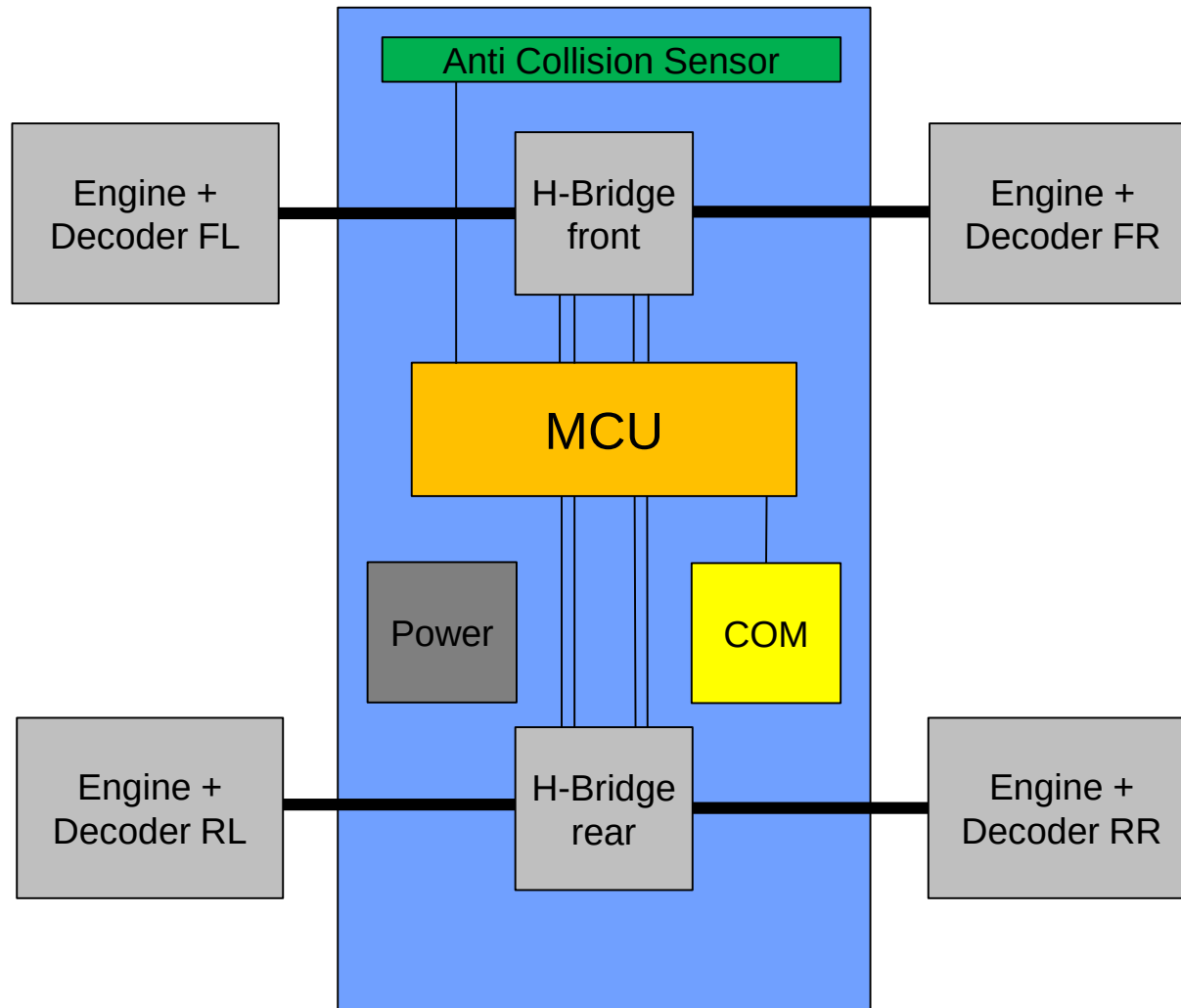


Risk management → feasibility study

Key question: Will the system work?
Hardware constraints are more painful than software issues!

Device	Checkpoint
Microcontroller	• Are the peripherals sufficient / compliant to the sensors / actors / communication systems • Is the CPU performance, memory (RAM / Flash / EEPROM) sufficient • Do we need specific safety / security features • Development tools available? Debugging support / features?
Sensors	• Reliability, resolution • Update rate
Actors	• Momentum, dynamics, speed
Communication	• Transfer rate, latency, jitter, reliability, overhead
Software libraries	• RTOS, driver, security libs, ... available or do we have to develop them
Power and environmental constraints	• Different voltage ranges / required current • Temperature range

High Level Block Diagram



Feasibility Check

Microcontroller choice

Proposal: Let's take the well-known PSOC5LP

Are the peripherals sufficient / compliant to the sensors / actors / communication systems

- 20 x Engine (1 PWM, 2 Mode, 2 Decoder per Engine)
- 2 x Communication (Rx, Tx)
- 2 x Anti-Collision Sensor (details tbd)

Is the CPU performance, memory (RAM / Flash / EEPROM) sufficient

- The PSOC5LP can be clocked with 72MHz → 72.000 cycles/ms
- 256kB Flash, 64kB RAM → Base system (OS, driver) requires around 10% Flash and 20% RAM

Do we need specific safety / security features

- Only on application level, not for the hardware itself

Development tools available? Debugging support / features?

- Yes

Peripheral Check

Engine Solution 1: Explicit pin for every signal and engine

- 20 pins required
- 4 PWM x 1
- 4 Decoder x 2 (A/B)
- 4 Mode x 2

What could be the limiting factor?

■ Digital peripherals

- Four 16-bit timer, counter, and PWM (TCPWM) blocks
- I²C, 1 Mbps bus speed
- USB 2.0 certified Full-Speed (FS) 12 Mbps peripheral interface (TID#10840032) using internal oscillator^[2]
- Full CAN 2.0b, 16 Rx, 8 Tx buffers
- 20 to 24 universal digital blocks (UDB), programmable to create any number of functions:
 - 8-, 16-, 24-, and 32-bit timers, counters, and PWMs
 - I²C, UART, SPI, I2S, LIN 2.0 interfaces
 - Cyclic redundancy check (CRC)
 - Pseudo random sequence (PRS) generators
 - Quadrature decoders
 - Gate-level logic functions

Design alternatives for the engine control

Alternative 1: control 2 engines (left, right) with the same PWM and use only 1 decoder

- Save 2 pins
- Comes with a loss of accuracy, as the engine rotation speed may be different with the same PWM (production faults, mechanical fitting,...)
- We cannot control the individual speed of the engine, e.g. detect a blocking engine – safety constraint

Alternative 2: use standalone motion controller

- Save 14 pins, as we can use CAN communication
- We have to use the provided control algorithms
- Costs will be significantly higher (around 1000€)



Communication / Software Libraries

Communication

- Bluetooth or Zigbee possible
- 38kBaud (or more?)
- Protocol length 5-10 bytes
- Max transfer frequency: 3ms
- Required: 25ms

Software Libraries

- RTOS and MCAL available



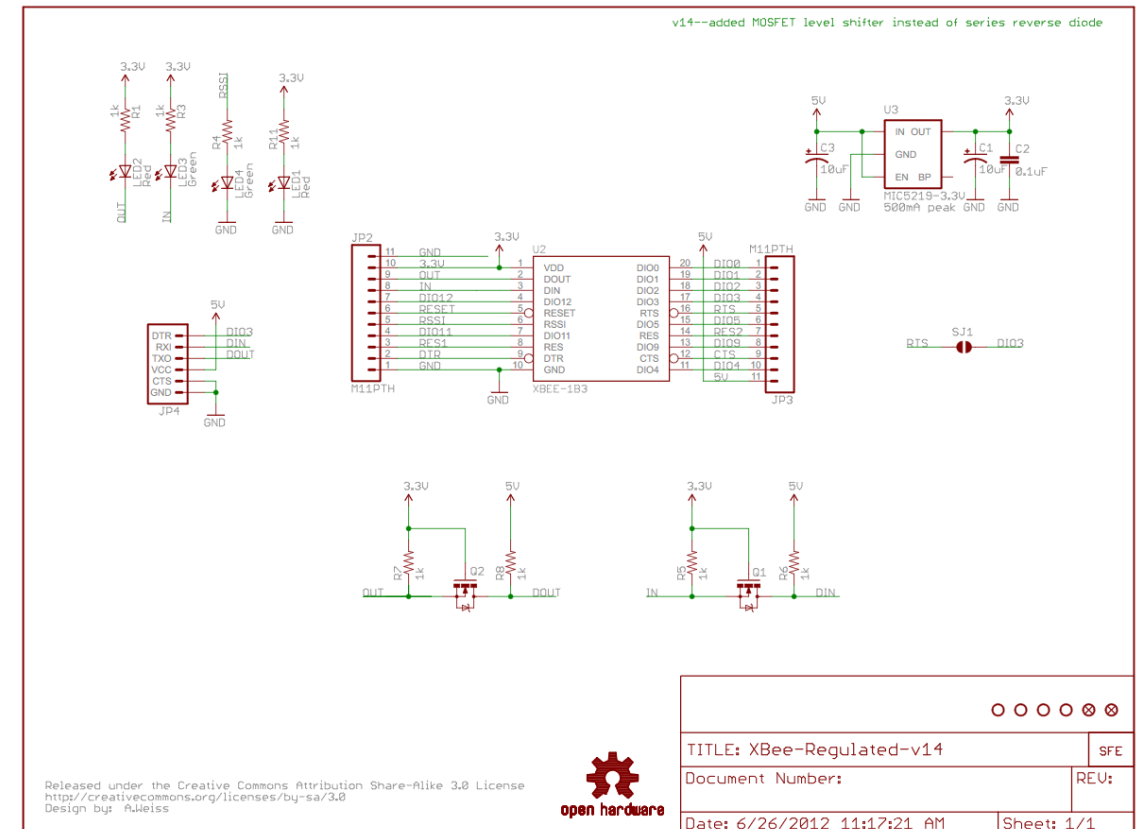
Power and Environmental requirements

Power domain:

- Engine: 6V
- PSOC: 1.7 – 5.5V
- Zigbee: 2.2 – 3.6V (DCDC on breakout)
- Battery: 7.2V

Environmental

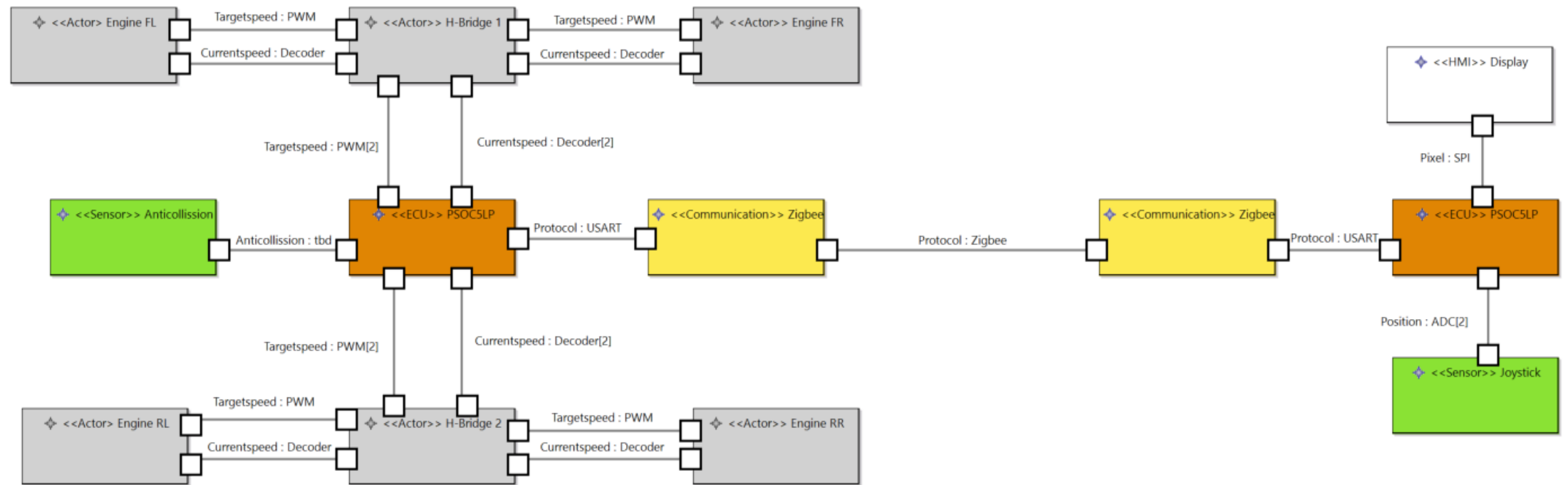
- No special requirements



Results feasibility check

Device		Checkpoint
Microcontroller		• Are the peripherals sufficient / compliant to the sensors / actors / communication systems
		• Is the CPU performance, memory (RAM / Flash / EEPROM) sufficient • Do we need specific safety / security features • Development tools available? Debugging support / features?
Sensors		• Reliability, resolution • Update rate
Actors		• Momentum, dynamics, speed
Communication		• Transfer rate, latency, jitter, reliability, overhead
Software libraries		• RTOS, driver, security libs, ... available or do we have to develop them
Power and environmental constraints		• Different voltage ranges / required current • Temperature range

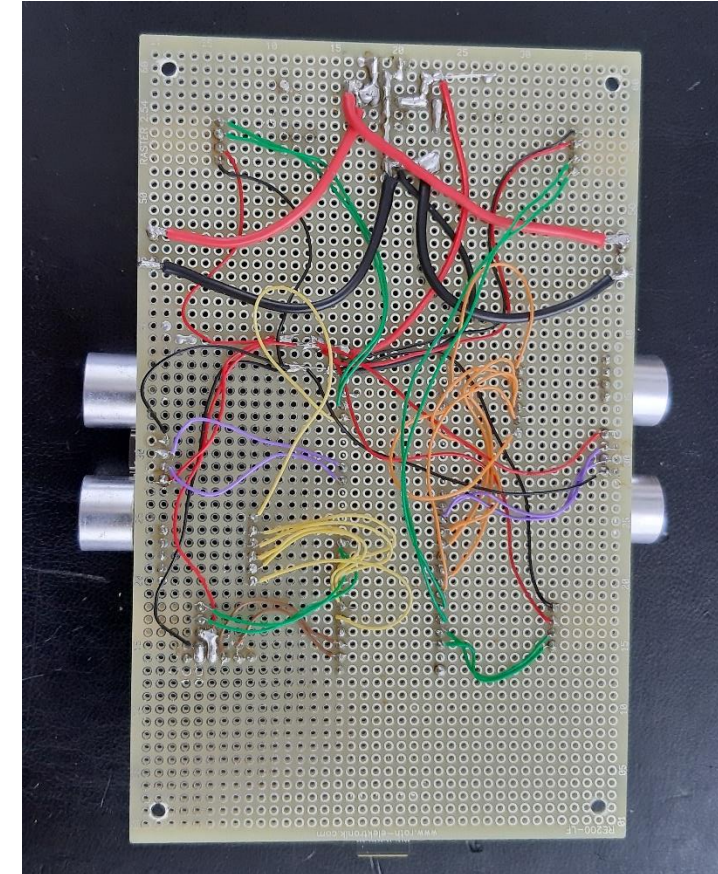
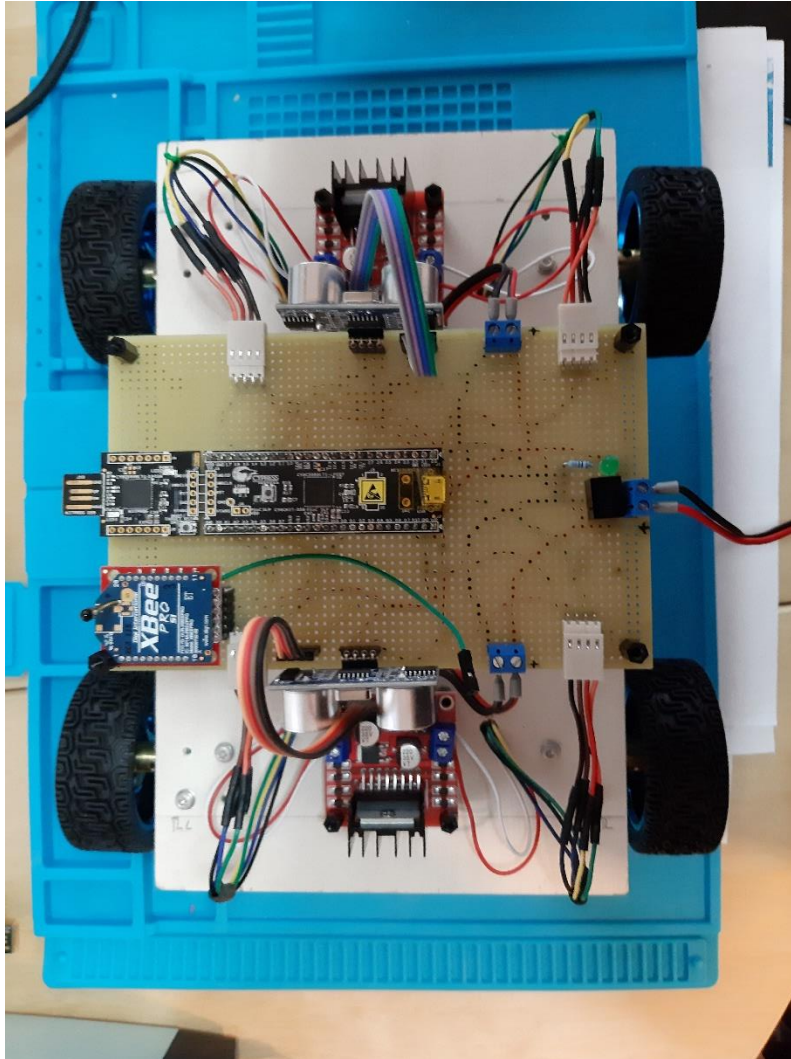
A more technical view / SysML Block Diagram



Identified risks / open questions from the hardware architecture

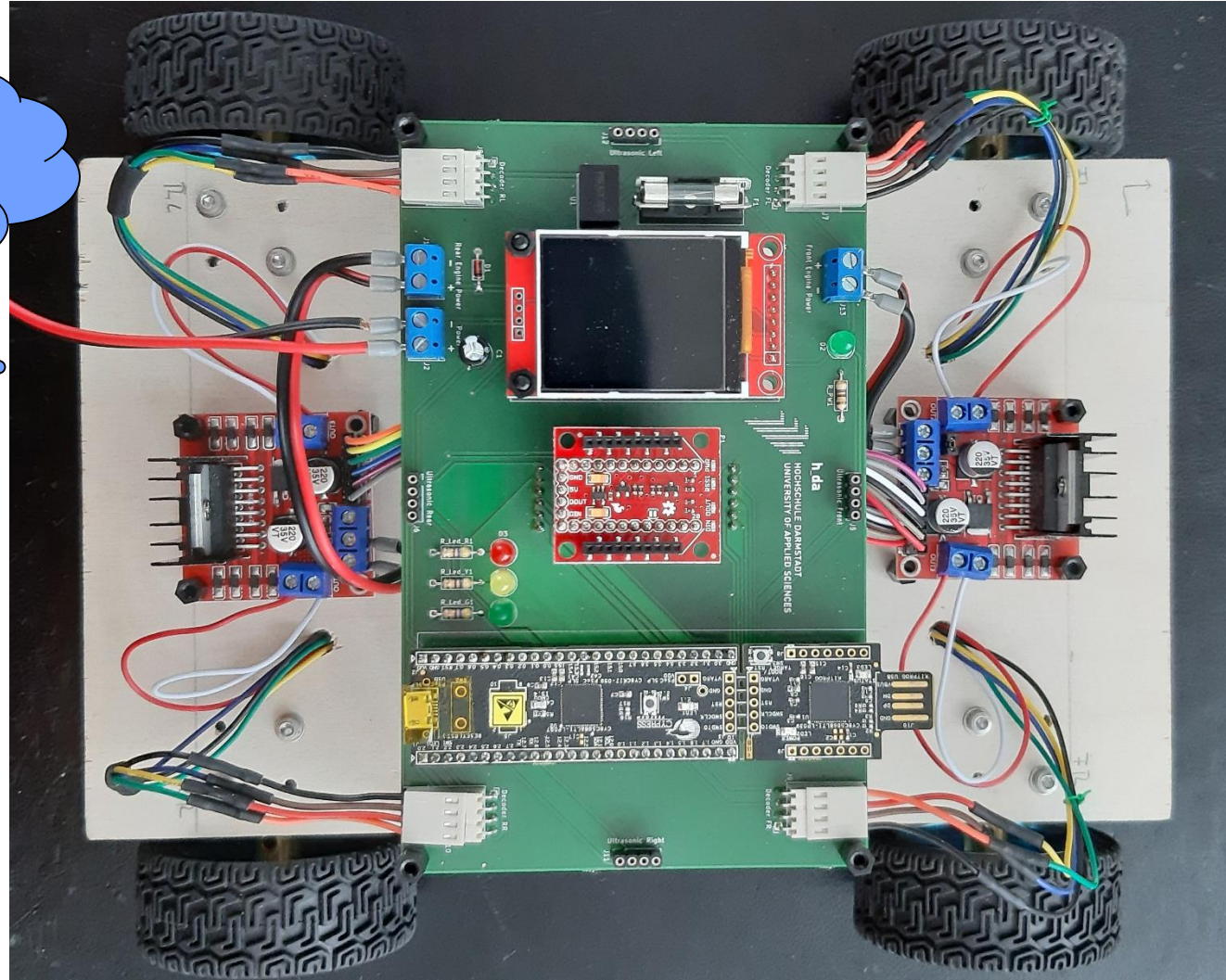
- We must check the available peripherals and UDB limitations especially for the engines.
- The resolution of the incremental decoder seems to be a bit low. We might have to develop an alternative (software) measurement principle especially for low speeds. A final design decision will be taken once we start implementing the engine controller logic.
- Concerning the anti collision sensor, we will discuss the technical concept later. As this is an own component, it will not interfere with the rest of the system (hopefully)
- We need different power domains.
 - Option 1: A power supply board providing different voltages
 - Option 2: A simple 5V DCDC converter and regulated break out boards e.g. for the XBEE

Hardware Skinny Sheep (Year 1)



Hardware Skinny Sheep (Year 2)

Unfortunately this
implies quite some
changes in the
software



On the other hand –
refactoring makes
sense...

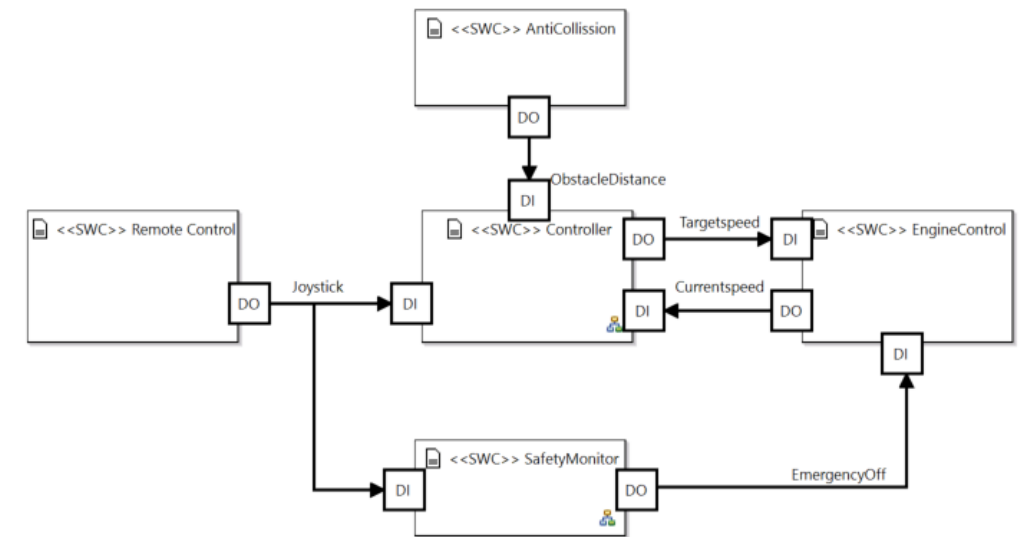
A first Software Architecture – car side

- We will use the OOA approach of “translating” real objects into SW objects.
- Then we have to design the signal flow between the objects (how this signal flow will be implemented will be discussed later)
- And we need to describe which execution units will be called when (multithreading)

The design still is a functional design – data and logic will be separated?
Pro's and con's?

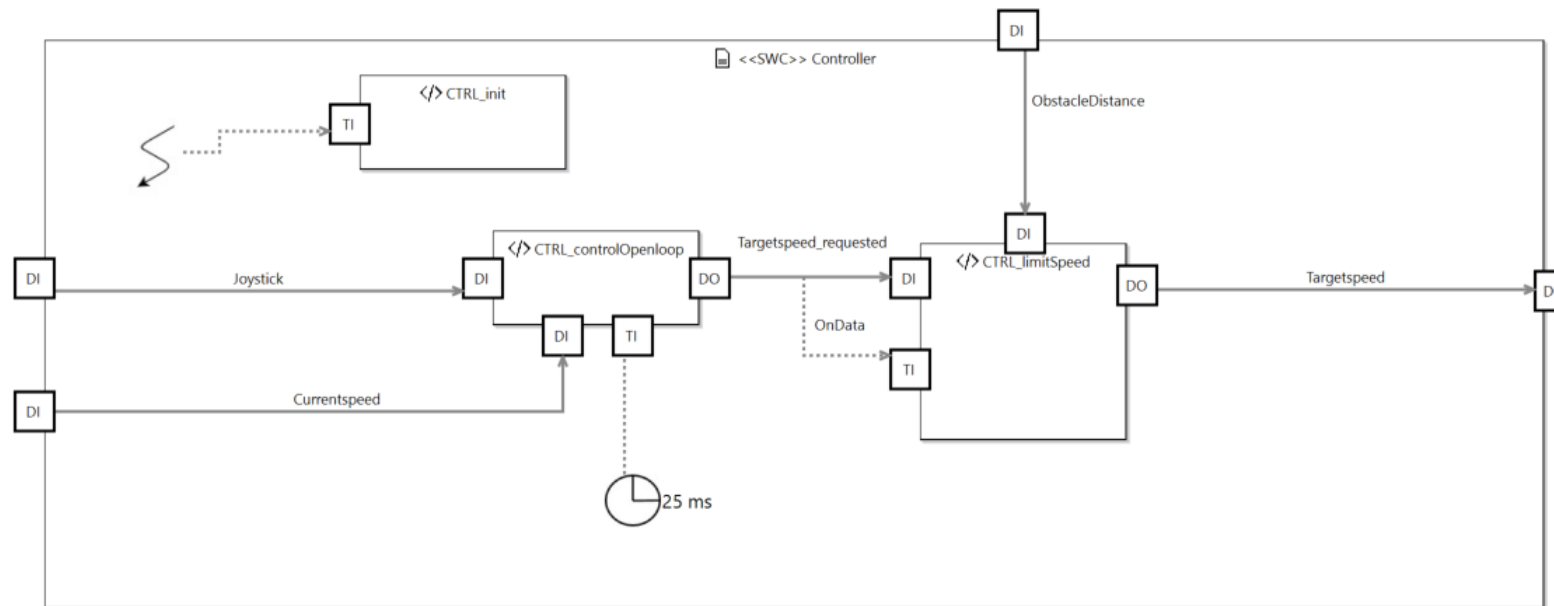
Software Architecture – High Level Component Diagram

- SWC or Software Components are Application Software Files representing a certain functionality
- SWC's use ports (DO / DI data out / data in) for communication
- Signals connect ports 1:m



Software Architecture – Detailed View

- Runnables are software functions which are triggered by the system
 - Cyclic
 - System state triggers (e.g. init)
 - On Data / On Error



Wrap-Up

- Creation of a first functional, hardware and software model
 - Understand what we want to do
 - Develop a first understanding of hardware and software
 - UML alone is not sufficient – will be used for detailed design later on
 - SysML block diagram (extension)
- Feasibility study focusing on hardware risks
 - Controller performance
 - Sensor reliability
 - Actor performance
 - Communication feasibility
 - Environmental aspects